

## 1. Objetivo

Durante el proyecto trabajaremos con **Scrum** y utilizaremos **GitHub** para almacenar y versionar toda la documentación y el código del proyecto.

La idea es que el repositorio no sea solamente un lugar donde "subimos archivos", sino que permita visualizar **cómo evoluciona el proyecto Sprint a Sprint**.

El proyecto tendrá:

- Product Backlog
- Épicas
- Features
- Historias de usuario
- Sprint Backlog
- Documentación de cada Sprint
- Código fuente, cuando corresponda
- Historial de cambios mediante commits

## 2. ¿Qué es una carpeta y qué es un archivo?

Es importante diferenciar ambos conceptos.

### Carpeta

Una carpeta sirve para **organizar varios archivos relacionados**.

### Ejemplo:

sprints/

Dentro pueden existir:

sprint-01/

sprint-02/

sprint-03/

### Archivo

Un archivo contiene información concreta.

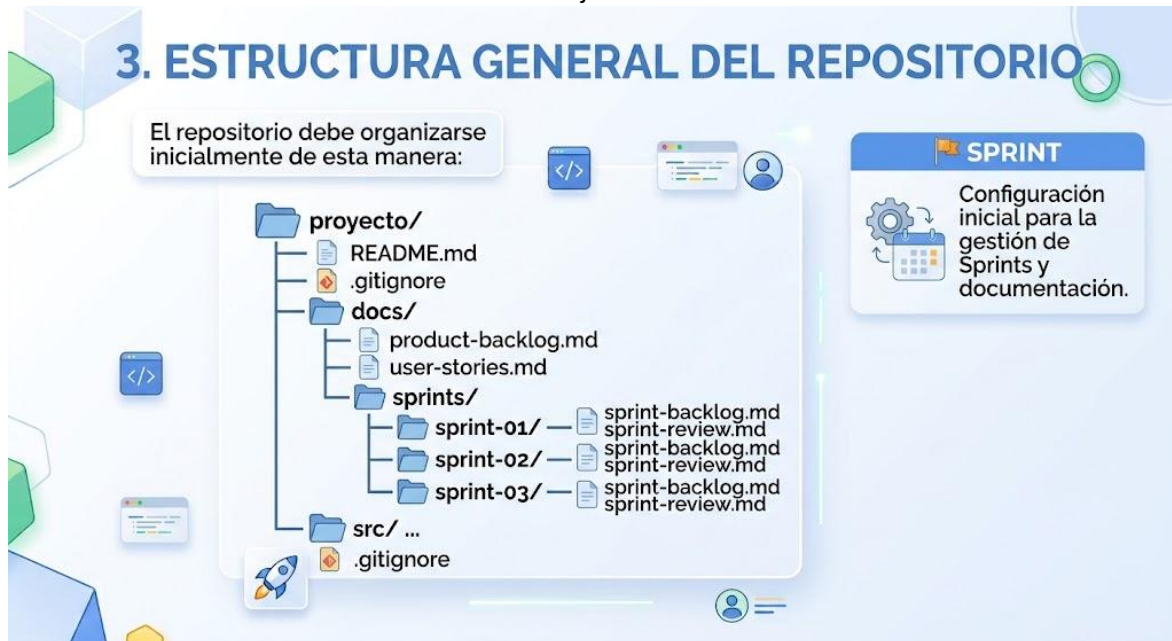
Por ejemplo:

product-backlog.md

sprint-backlog.md

sprint-review.md

La extensión .md significa **Markdown**.



**No es necesario crear todas las carpetas de los Sprints desde el principio.**

Al comenzar solamente necesitamos:

proyecto/

├── README.md

├── docs/

│ ├── product-backlog.md

│ └── user-stories.md

└── sprints/

src/

└── .gitignore

Cuando comience el Sprint 1, se crea:

sprints/

└── sprint-01/

Cuando llegue el Sprint 2:

sprints/

├── sprint-01/

└── sprint-02/

Y así sucesivamente.

#### 4. ¿Qué información va en cada archivo?

##### **README.md**

Es la portada del proyecto.

Debe permitir que una persona que entra al repositorio entienda rápidamente:

- Qué problema resuelve el proyecto.
- Qué producto se está desarrollando.
- Quiénes integran el equipo.
- Tecnologías utilizadas.
- Cómo está organizado el repositorio.
- Estado actual del proyecto.

Ejemplo:

# Nombre del proyecto

Descripción breve del producto.

## Equipo

- Nombre estudiante 1
- Nombre estudiante 2
- Nombre estudiante 3

## Tecnologías

- Tecnología 1
- Tecnología 2

## Organización

El proyecto se desarrolla utilizando Scrum  
y se organiza en diferentes Sprints.

#### 5. Product Backlog

El Product Backlog representa **todo lo que potencialmente necesita el producto**.

Es un documento permanente.

Por ejemplo:

docs/

└─ product-backlog.md

Aquí estarán las:

- Épicas
- Features
- Historias de usuario
- Prioridades
- Estimaciones
- Estado

**Ejemplo:**

# Product Backlog

## Épica 1: Gestión de usuarios

### Feature 1.1: Registro

- US-01: Como usuario quiero registrarme  
para poder utilizar la plataforma.
- US-02: Como usuario quiero recuperar mi contraseña  
para poder acceder nuevamente.

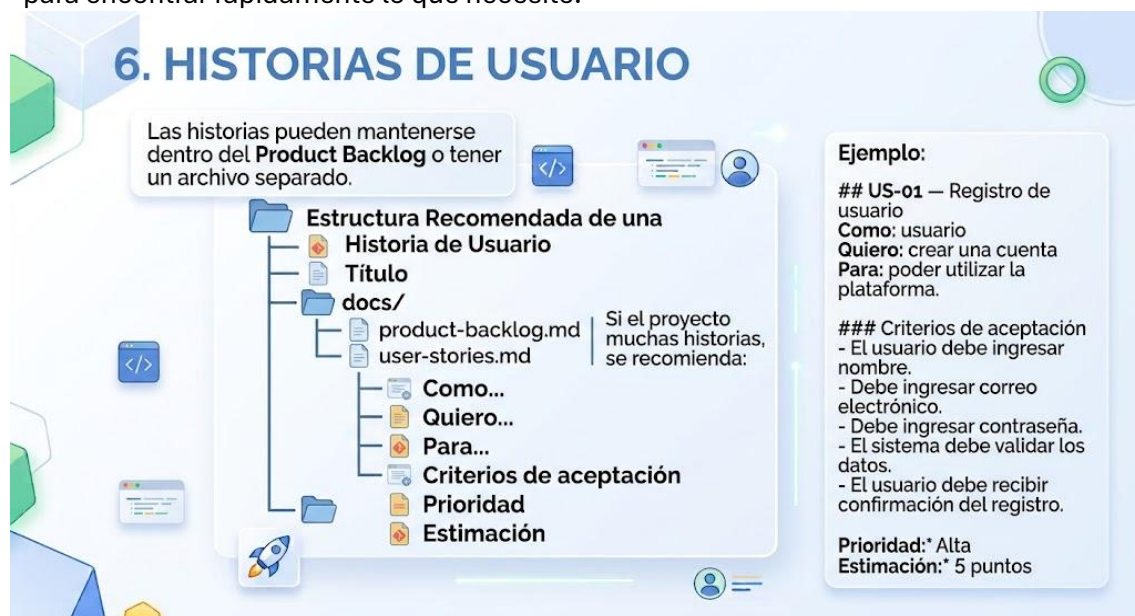
### Feature 1.2: Inicio de sesión

- US-03: Como usuario quiero iniciar sesión para acceder a mi cuenta.

## Épica 2: Gestión de productos

### Feature 2.1: Catálogo

- US-04: Como usuario quiero visualizar productos para conocer las opciones disponibles.
- US-05: Como usuario quiero buscar productos para encontrar rápidamente lo que necesito.



## 7. ¿Qué es el Sprint Backlog?

El Sprint Backlog contiene **solamente el trabajo seleccionado para ese Sprint**.

Por eso cada Sprint tiene su propio archivo.

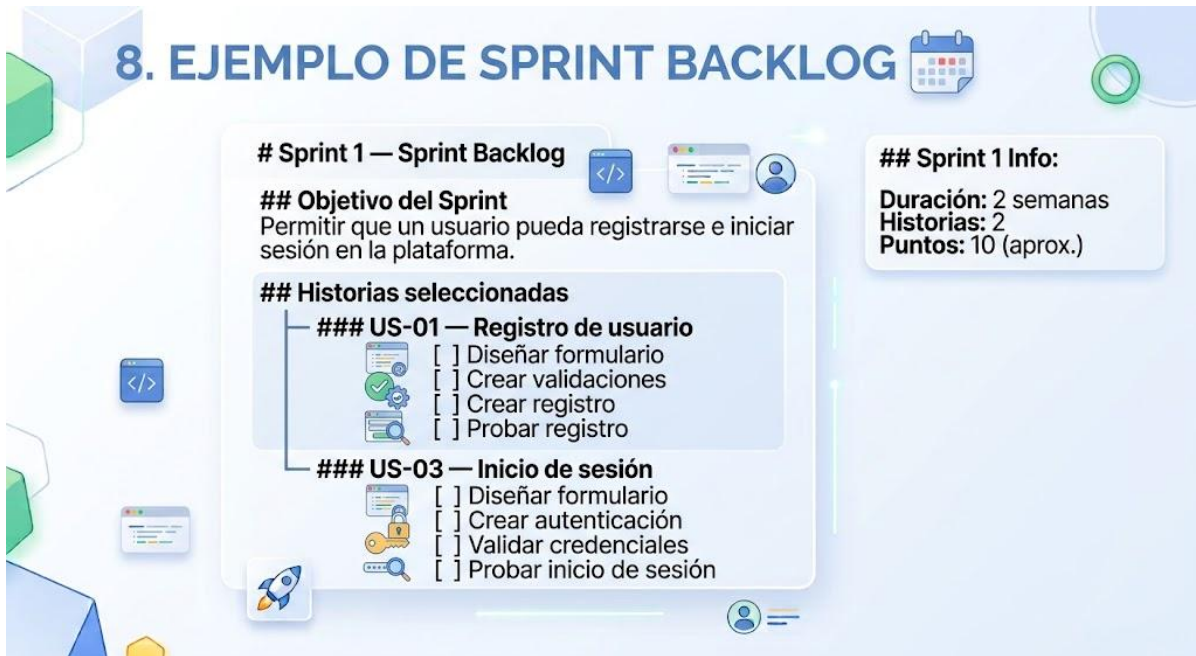
Ejemplo:

```
docs/  
├── sprints/  
│   └── sprint-01/  
│       └── sprint-backlog.md
```

Para Sprint 2:

```
docs/  
├── sprints/  
│   └── sprint-02/  
│       └── sprint-backlog.md
```

El Product Backlog continúa siendo uno solo.



### 9. ¿Qué es sprint-review.md?

Es una pequeña documentación del cierre del Sprint.

Por ejemplo:

# Sprint 1 — Sprint Review

## Objetivo

Implementar el registro y el inicio de sesión.

## Historias completadas

- US-01

- US-03

## Historias no completadas

- Ninguna

## Resultado

Se logró implementar el flujo básico de registro e inicio de sesión.

Este archivo permite conservar evidencia de lo ocurrido en cada Sprint.

### 10. ¿Qué hacemos con los archivos actuales de Word?

No es necesario volver a escribir todo desde cero.

Los documentos que actualmente están en Word deben convertirse a **Markdown (.md)**.

Por ejemplo:

Product Backlog.docx

se convierte en:

product-backlog.md

Y:  
Sprint Backlog.docx  
se convierte en:  
sprint-backlog.md

---

## 11. ¿Qué es Markdown?

Markdown es una forma sencilla de escribir documentos utilizando texto y algunos símbolos.

Por ejemplo:

# Título

## Subtítulo

**\*\*Texto en negrita\*\***

- Elemento 1

- Elemento 2

- Elemento 3

GitHub interpreta automáticamente este formato y lo muestra como un documento visual.  
No necesitan utilizar Word para estos documentos.

## 12. ¿Cómo crear un archivo .md?

Existen varias formas.

### Opción 1 — Desde GitHub

En el repositorio:

1. Seleccionar **Add file**.
2. Seleccionar **Create new file**.
3. Escribir:

docs/product-backlog.md

4. Escribir el contenido.
5. Guardar el cambio.

GitHub creará automáticamente la carpeta docs si todavía no existe.

### Opción 2 — Desde Visual Studio Code

Crear una carpeta llamada:

docs

Dentro crear:

product-backlog.md

Luego abrir el archivo y escribir el contenido en Markdown.

Visual Studio Code permite visualizar el resultado mediante la vista previa de Markdown.

### Opción 3 — Utilizar Inteligencia Artificial

También pueden utilizar la **IA de su preferencia** para convertir los documentos que actualmente tienen en Word a formato Markdown (.md).

Por ejemplo, pueden cargar el archivo Word en una herramienta de IA y solicitar:

**"Convierte este documento a formato Markdown (.md), manteniendo toda la información, títulos, tablas, listas y estructura del contenido."**

Luego:

1. Revisen el resultado generado por la IA.
2. Verifiquen que **no se haya perdido información**.
3. Corrijan cualquier error de formato.
4. Copien el contenido al archivo correspondiente, por ejemplo:

docs/product-backlog.md

o

docs/user-stories.md

**Importante:** la IA ayuda a convertir y organizar el contenido, pero **ustedes son responsables de revisar que el resultado sea correcto**. No se trata de subir lo que genere la IA sin revisarlo.

### 13. ¿Qué pasa con el formato de Word?

No intenten copiar exactamente el diseño visual de Word.

En Markdown importa principalmente:

- La estructura.
- Los títulos.
- Las tablas.
- Las listas.
- La información.
- La legibilidad.

GitHub se encargará de mostrar el documento de manera ordenada.

### 14. Commits: ¿cómo deben trabajar?

Un commit representa un **cambio concreto y coherente**.

Eviten commits como:

subiendo trabajo

trabajo final

cosas del sprint

ejercicio completo

Es preferible utilizar mensajes claros.

Ejemplos:

chore: initialize repository structure

docs: add product vision

docs: define project epics

docs: add product backlog features

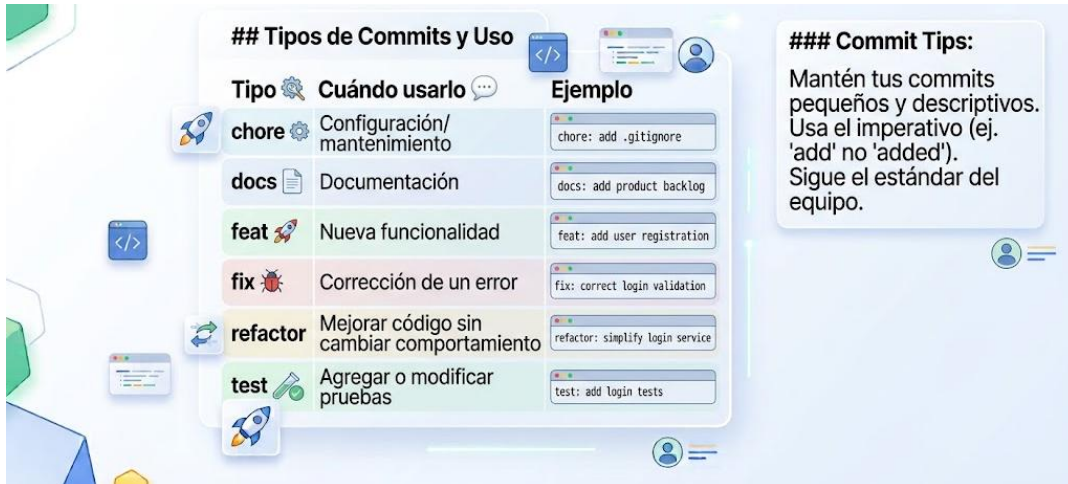
docs: add user stories

docs: define sprint 1 backlog

docs: update sprint 1 acceptance criteria

La idea es que al revisar el historial de GitHub se pueda entender cómo evolucionó el proyecto.

### Posibles categorías de commits:



### 15. Orden recomendado para el Sprint 1

Durante el trabajo inicial pueden realizar commits en este orden:

1. Crear estructura inicial  
↓
2. Agregar README  
↓
3. Agregar visión del producto  
↓
4. Agregar épicas  
↓
5. Agregar features  
↓
6. Agregar historias de usuario  
↓
7. Completar Product Backlog  
↓
8. Seleccionar historias para Sprint 1  
↓
9. Crear Sprint Backlog  
↓
10. Refinar y corregir

No es obligatorio hacer exactamente diez commits, pero cada cambio importante debería quedar registrado.



### 16. ¿Qué ocurre cuando termina el Sprint 1?

El proyecto NO empieza nuevamente.

El mismo repositorio continúa.

Por ejemplo:

Sprint 1

↓

Revisión

↓

Actualizar Product Backlog

↓

Planificar Sprint 2

↓

Crear sprint-02

↓

Crear nuevo Sprint Backlog

↓

Desarrollar Sprint 2

El Product Backlog sigue siendo el mismo archivo:

docs/product-backlog.md

Mientras que cada Sprint tiene su propio espacio:

docs/sprints/sprint-01/

docs/sprints/sprint-02/

docs/sprints/sprint-03/

### 17. ¿Qué pasa con las historias que no terminaron?

No se eliminan.

Una historia que no se completó debe volver al Product Backlog y posteriormente puede ser considerada para otro Sprint.

Por ejemplo:

Product Backlog

US-01 → Done

US-02 → Done

US-03 → To Do

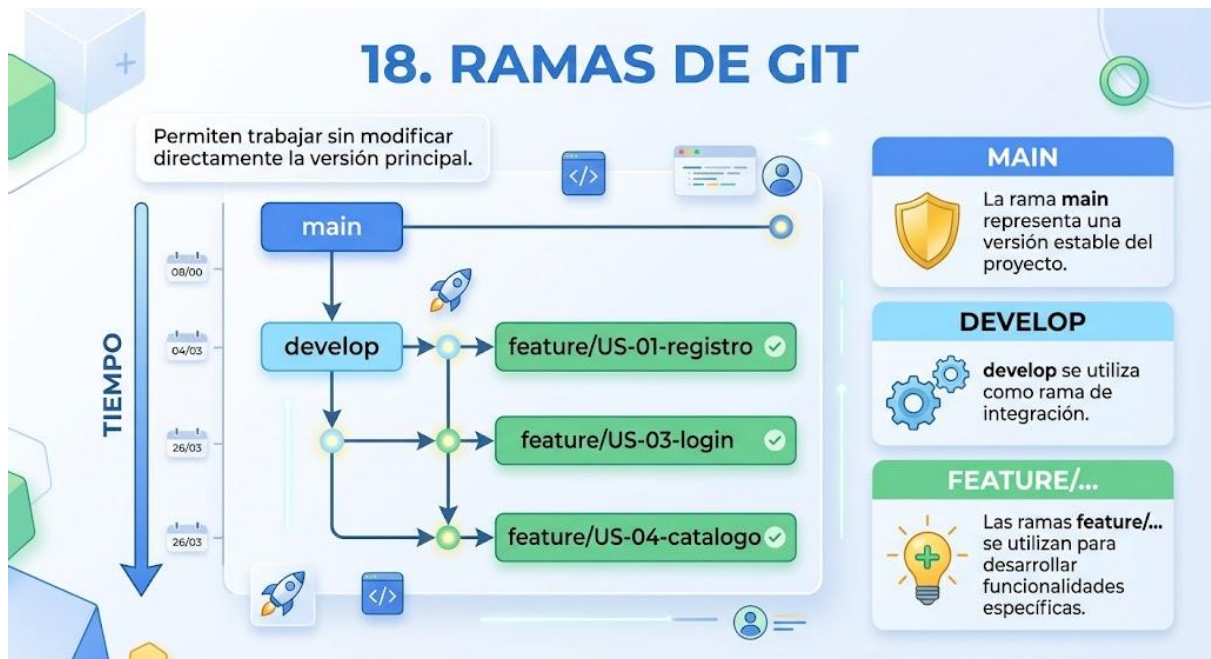
US-04 → To Do

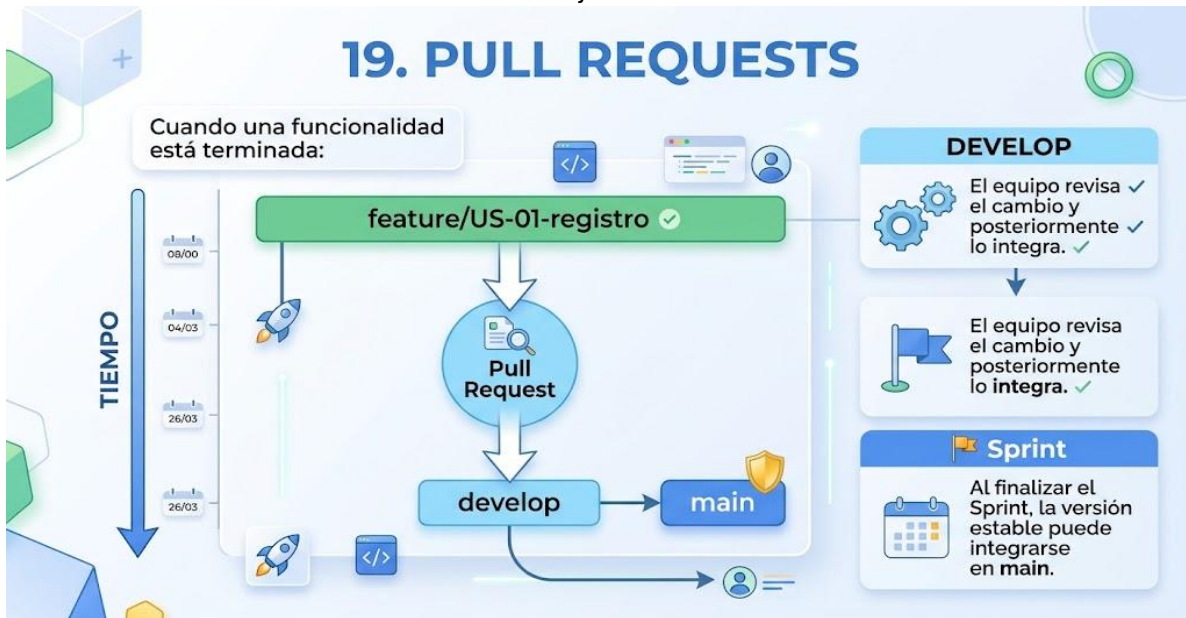
En Sprint 2 podrían seleccionarse:

US-03

US-04

US-05





### 20. Versiones por Sprint

Al finalizar cada Sprint pueden identificar una versión del producto.

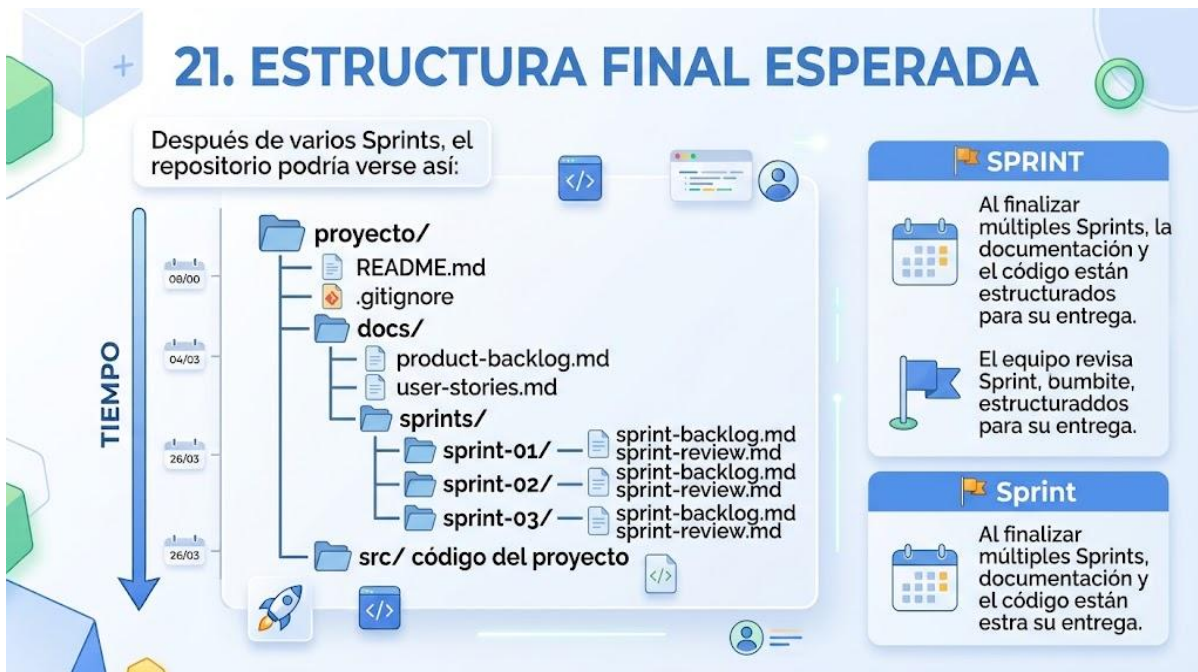
Por ejemplo:

Sprint 1 → v0.1.0

Sprint 2 → v0.2.0

Sprint 3 → v0.3.0

Esto permite identificar qué versión del producto corresponde a cada Sprint.



## **22. Regla fundamental**

Recuerden estas cuatro reglas:

- 1. Un solo repositorio para todo el proyecto.**
- 2. Un Product Backlog que evoluciona durante todo el proyecto.**
- 3. Un Sprint Backlog diferente para cada Sprint.**
- 4. Commits pequeños, claros y relacionados con un cambio concreto.**

El objetivo no es solamente "tener los archivos en GitHub".

El objetivo es que cualquier persona pueda entrar al repositorio y entender:

**qué producto estamos construyendo, qué queremos construir, qué hicimos en cada Sprint y cómo ha evolucionado el proyecto.**

## **23. Checklist para entregar el Sprint 1**

Antes de considerar terminada la primera entrega, verificar:

- Existe el repositorio en GitHub.
- Existe README.md.
- Existe la carpeta docs.
- Product Backlog está en formato .md.
- Las épicas están definidas.
- Las features están asociadas a las épicas.
- Las historias de usuario tienen identificador.
- Las historias tienen criterios de aceptación.
- Está definido el Sprint Backlog.
- Los documentos están organizados en carpetas.
- Los cambios importantes tienen commits.
- Los nombres de los commits son claros.
- El repositorio no contiene solamente un archivo Word final.
- El equipo puede explicar cómo continuará con el Sprint 2.

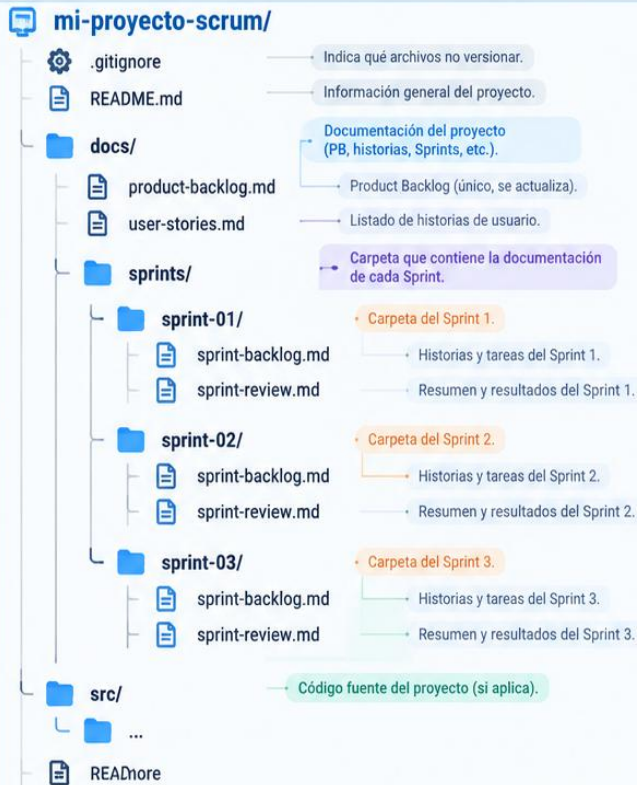
ESCUELA DE TECNOLOGIA Y ADMINISTRACION  
Técnico Laboral como Asistente en Desarrollo de Software  
Submódulo: Metodologías Ágiles  
Proyecto Integrador: Aplicación de Scrum en Proyecto Propio  
Manual de trabajo: Scrum + GitHub



## Estructura del repositorio - Proyecto Scrum

Organiza tu trabajo, documenta cada Sprint y mantén un historial claro en GitHub.

Un buen repositorio  
cuenta la historia  
de tu proyecto



### Archivos principales

- README.md** → descripción del proyecto, equipo, equipo, tecnologías, cómo se organiza el repo, estado actual.
- .gitignore** → evita subir archivos innecesarios.
- product-backlog.md** → épicas, features, historias, prioridades, estimaciones, estado.
- user-stories.md** → detalle de historias de usuario (ID, descripción, criterios de aceptación, etc.).



### Carpeta docs/

- Es la carpeta donde se guarda toda la documentación en formato .md.
- Contiene el Product Backlog y la información de cada Sprint.



### Carpeta sprints/

- Cada Sprint tiene su propia carpeta.
- Dentro de cada carpeta encontrarás:
  - sprint-backlog.md** → tareas y historias del Sprint.
  - sprint-review.md** → qué se hizo, qué no, resultados.



### Carpeta src/

- Aquí va el código del proyecto (si aplica).
- Puede contener subcarpetas (ej: src/components, src/pages, etc.).



### Importante

- El Product Backlog es un solo archivo que se actualiza en todo si proyecto.
- Cada Sprint tiene su propio Sprint Backlog y su propio Sprint Review.
- No dupliques información, mantén la estructura.
- Usa Markdown (.md) en lugar de Word.



### Ramas de Git (flujo recomendado)



### Ejemplos de commits

- chore: initialize repository structure
- docs: add product vision
- docs: define project epics
- docs: add product backlog features
- docs: add user stories
- docs: define sprint 1 backlog
- docs: update sprint 1 acceptance criteria
- feat: implement login feature
- docs: add sprint 1 review
- chore: prepare for sprint 2



### Versiónes por Sprint



Recuerda: Un solo repositorio, un Product Backlog permanente, un Sprint Backlog por Sprint y commits claros y pequeños.

¡Éxitos en su proyecto!

## **24. Próximo paso: llevar el proyecto a Jira**

En las siguientes clases vamos a incorporar **Jira** al proyecto para gestionar el trabajo utilizando Scrum.

Hasta este momento, estamos organizando la información y aprendiendo a trabajar correctamente con **GitHub**. Posteriormente, esa misma información será trasladada a Jira para gestionar el proyecto de manera más visual y cercana a un entorno profesional.

### **¿Qué tendremos en GitHub?**

GitHub será principalmente el lugar donde estará:

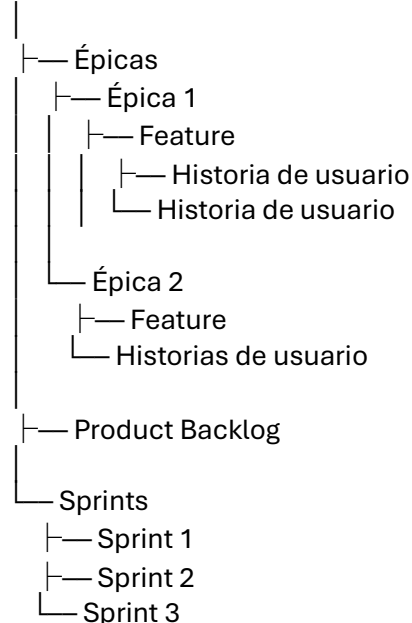
- El código fuente.
- La documentación del proyecto.
- El Product Backlog documentado.
- Los Sprint Backlogs.
- La información histórica de cada Sprint.
- El historial de cambios mediante commits.

### **¿Qué tendremos en Jira?**

Jira será utilizado para **gestionar y visualizar el trabajo Scrum**.

La información se organizará aproximadamente así:

Jira



**¿Qué vamos a hacer en las próximas clases?**

Partiremos de lo que ustedes ya construyeron.

**DOCUMENTACIÓN EN GITHUB**

↓  
 Épicas  
 ↓  
 Features  
 ↓  
 Historias de usuario  
 ↓  
 Product Backlog  
 ↓  
 JIRA  
 ↓  
 Product Backlog en Jira  
 ↓  
 Planificación del Sprint  
 ↓  
 Sprint Backlog  
 ↓  
 Tablero Scrum  
 ↓  
 Sprint 1



No deberán crear nuevamente el proyecto desde cero.

**Vamos a tomar el trabajo realizado y aprender a gestionarlo dentro de Jira.**

**¿Por qué utilizaremos las dos herramientas?**

Porque cumplen funciones diferentes:

| Herramienta   | Principal utilización                                  |
|---------------|--------------------------------------------------------|
| <b>GitHub</b> | Código, documentación, versiones y control de cambios  |
| <b>Jira</b>   | Planificación, seguimiento y gestión del trabajo Scrum |

En un entorno profesional es habitual que las herramientas estén conectadas. Por ejemplo, una historia de usuario en Jira puede estar relacionada con una rama, un commit o un Pull Request de GitHub.

**Importante**

**No dupliquen trabajo innecesariamente.**

Por ahora, continúen organizando correctamente la documentación y el repositorio en GitHub. En las próximas clases veremos cómo transformar esa estructura en un proyecto Scrum dentro de Jira y cómo relacionar el trabajo de Jira con GitHub.

**Primero aprendemos a organizar el trabajo.**

**Después aprendemos a gestionarlo profesionalmente.**